

VITE&Gourmand Documentation Technique

Luidgi
NICOLAS

Sommaire

1. Réflexions initiales technologiques	3
2. Environnement de travail.....	6
3. MCD / Diagramme de classes	8
4. Diagramme d'utilisation et diagrammes de séquence.....	10
5. Déploiement Heroku démarche et étapes	12

1. Réflexions initiales technologiques

1.1 Back-end : PHP natif (architecture MVC) vs framework (Symfony/Laravel)

Le projet VITEGourmand est développé en PHP 8.2 natif, avec une architecture MVC construite à la main plutôt qu'avec un framework comme Symfony ou Laravel.

Front controller unique : toutes les requêtes passent par public/index.php, qui route les pages via un switch sur le paramètre ?page= (ex. ?page=commander, ?page=admin_dashboard). Chaque route charge explicitement son contrôleur et ses modèles avant de rendre une vue via BaseController::render().

Justification du choix :

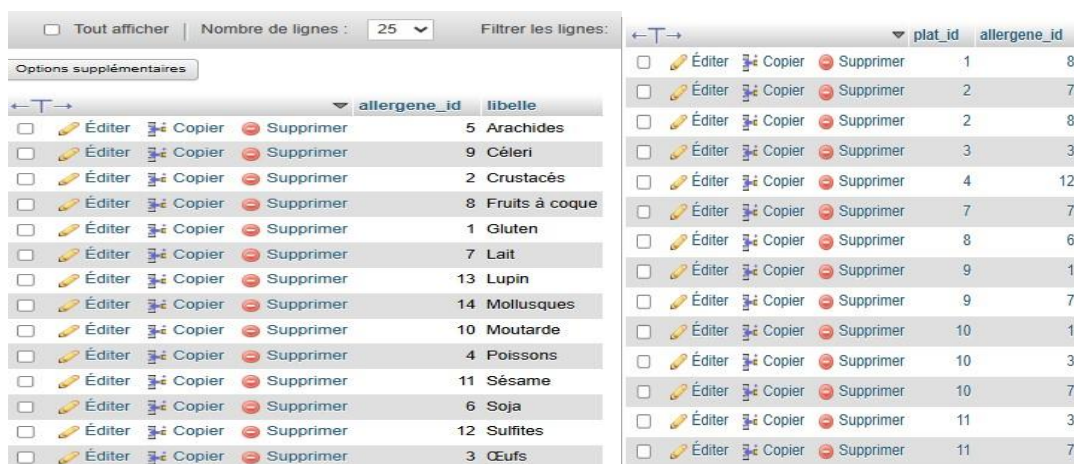
- Maîtrise complète du cycle requête → contrôleur → modèle → vue, sans « magie » de framework à apprendre en parallèle du reste du programme (contrainte de temps ECF).
- Légèreté : aucune dépendance lourde, démarrage immédiat sous XAMPP sans configuration de framework.
- Compatibilité simple avec Heroku (buildpack PHP standard, pas de cache/config framework à gérer en production).

Un framework (Symfony/Laravel) aurait apporté routing, ORM et sécurité « prêts à l'emploi », au prix d'une courbe d'apprentissage incompatible avec le délai imparti ; le choix natif a permis de concentrer l'effort sur la logique métier (allergènes, délais de commande, zones de livraison) plutôt que sur la prise en main d'un framework

1.2 Base de données hybride : MySQL + MongoDB Atlas

Le sujet impose l'usage conjoint d'une base relationnelle et d'une base non relationnelle. Ce choix a été mis à profit plutôt que subi :

- MySQL porte les données structurées et relationnelles : utilisateurs, rôles, plats, menus, allergènes (relation N-N plat_allergene), commandes, suivi de commande. Voir database/schema.sql.



allergene_id	libelle
5	Arachides
9	Céleri
2	Crustacés
8	Fruits à coque
1	Gluten
7	Lait
13	Lupin
14	Mollusques
10	Moutarde
4	Poissons
11	Sésame
6	Soja
12	Sulfites
3	Œufs

plat_id	allergene_id
1	8
2	7
2	8
3	3
4	12
7	7
8	6
9	1
9	7
10	1
10	3
10	7
11	3
11	7

- MongoDB Atlas porte les données peu structurées et à schéma variable : avis clients (AvisModel, app/models/AvisModel.php) et statistiques de commandes



- (StatsCommandeModel). Un avis n'a pas besoin d'un schéma rigide (statut de modération, note, commentaire libre) et se prête bien à un document JSON.

Justification : séparer les deux évite de forcer des données non structurées (avis en modération, statuts variables) dans un schéma SQL rigide, tout en gardant l'intégrité référentielle (clés étrangères, transactions PDO) là où elle est indispensable notamment sur la commande, où une transaction garantit que la décrémentation de stock et l'enregistrement de la commande restent cohérents

1.3 Envoi d'emails : PHPMailer + Mailtrap vs SMTP Gmail

L'envoi d'emails (confirmation de commande, réinitialisation de mot de passe) utilise PHPMailer configuré en SMTP sur Mailtrap, un sandbox qui intercepte les emails sans les délivrer réellement.

Pourquoi Mailtrap plutôt que Gmail SMTP : un compte Gmail expose un mot de passe applicatif réel et un risque de blocage du compte (limites d'envoi, détection antispam) en environnement de test/développement où de nombreux emails de test sont envoyés. Mailtrap est conçu pour cet usage : aucun risque d'envoyer un email de test à un vrai destinataire, inspection facile du contenu HTML envoyé.

En production (Heroku), seule la variable d'environnement MAILTRAP_* changerait pour un vrai fournisseur SMTP (ex. SendGrid, Gmail applicatif) le code de mailer.php n'a pas besoin d'être modifié, seule la configuration change (voir section 5, Déploiement).

1.4 Front-end : HTML5 + Bootstrap + JavaScript vanille vs React

Le front-end repose sur du HTML5 serveur (vues PHP), Bootstrap pour la grille et les composants, du JavaScript vanille pour les interactions dynamiques (AJAX zone de livraison, validation de formulaire), et la police Poppins (commit 645e01a).

Justification : le projet ne nécessite pas d'application monopage (SPA) — les échanges dynamiques se limitent à quelques appels AJAX ciblés (vérification de zone de livraison, gestion admin). React aurait ajouté une étape de build, un découpage en composants et une gestion d'état, sans bénéfice réel pour ce périmètre, et aurait complexifié le déploiement Heroku (build front séparé). Bootstrap permet un rendu mobile-first cohérent avec un minimum de CSS custom.

1.5 Hébergement : Heroku vs OVH / hébergement mutualisé

L'application est déployée sur Heroku via le buildpack PHP officiel (Procfile : web: heroku-php-apache2 public/).

Justification : Heroku permet un déploiement par git push sans administration serveur (pas de configuration Apache/PHP manuelle), avec des add-ons prêts à l'emploi pour la base MySQL (JawsDB) et des variables d'environnement centralisées pour les secrets. Un hébergement mutualisé (OVH) aurait nécessité une configuration manuelle (FTP, phpMyAdmin, php.ini) plus longue à mettre en place et moins traçable, pour un gain de coût non déterminant dans le cadre d'un ECF.

1.6 Contraintes prises en compte

- RGPD / allergènes (règlement UE n° 1169/2011) : ce règlement impose l'information du consommateur sur les 14 allergènes majeurs. Le modèle de données prévoit une table allergene dédiée et une relation N-N avec plat, permettant d'afficher et de filtrer les plats par allergène (voir 1.2). Le RGPD à proprement parler encadre par ailleurs les données personnelles des utilisateurs (nom, adresse, téléphone) : celles-ci ne sont utilisées que pour la gestion des commandes et ne sont pas partagées avec des tiers.
- Délais événementiels : les menus sont pensés pour des commandes anticipées (traiteur événementiel), d'où la présence d'un délai de commande minimum configurable par menu (delai_commande_valeur / delai_commande_unite) qui bloque réellement la date de prestation choisie si le délai n'est pas respecté (voir app/controllers/commandeController.php:127-135).
- Mobile-first : chaque vue déclare une balise viewport (app/views/includes/header.php:5) et s'appuie sur la grille responsive de Bootstrap, pour un affichage utilisable aussi bien sur smartphone que sur desktop.

2. Environnement de travail

2.1 Outils utilisés

Le développement s'est fait en local sous XAMPP (Apache + MySQL + PHP 8.2.12), avec Composer pour la gestion des dépendances PHP, VS Code comme éditeur, et phpMyAdmin pour l'administration de la base MySQL locale.

2.2 Extensions et dépendances PHP requises

Les prérequis techniques sont déclarés dans composer.json :

```
{
  "require": {
    "php": ">=8.1.0",
    "ext-curl": "*",
    "ext-mongodb": "*",
    "ext-pdo_mysql": "*",
    "ext-mbstring": "*",
    "mongodb/mongodb": "^2.3",
    "phpmailer/phpmailer": "^7.1"
  }
}
```

Point d'attention spécifique à XAMPP : ext-mongodb est le pilote PHP permettant de dialoguer avec MongoDB il ne s'agit pas d'un serveur MongoDB à installer en local. Le projet utilise en effet MongoDB Atlas, un cluster MongoDB hébergé dans le cloud (voir app/config/mongo.php et les variables MONGO_* en 2.3) : aucune installation ni administration de serveur MongoDB n'est donc nécessaire en local, seule la connexion réseau vers Atlas est requise.

Ce pilote n'est en revanche pas inclus par défaut dans une installation XAMPP standard. Il faut télécharger le fichier php_mongodb.dll correspondant à la version de PHP utilisée et l'activer manuellement dans php.ini (extension=mongodb), sans quoi l'application plante au chargement de tout contrôleur touchant aux avis ou aux statistiques (connexion à Atlas impossible sans le pilote).

2.3 Configuration locale : séparation code / secrets

Deux fichiers de configuration sensibles sont exclus du dépôt Git (voir .gitignore) et doivent être créés localement à partir d'un exemple versionné :

Fichier réel (gitignoré)	Exemple versionné	Contenu
.env	.env.example	Identifiants Mailtrap (MAILTRAP_*) et MongoDB Atlas (MONGO_*)
app/config/constants.local.php	app/config/constants.local.php.example	Identifiants MySQL locaux (DB_USER, DB_PASS)

Ce choix évite qu'un identifiant de connexion ne soit committé, même par erreur (voir aussi le correctif de sécurité du commit da08c0d, qui a retiré des identifiants en dur du code pour les déplacer dans le fichier gitigniré).

Le mécanisme est renforcé côté code : app/config/constants.php:19-24 vérifie l'existence de constants.local.php avant toute connexion et arrête l'application avec un message explicite si le fichier est absent, plutôt que de laisser échouer silencieusement une connexion PDO avec des identifiants vides :

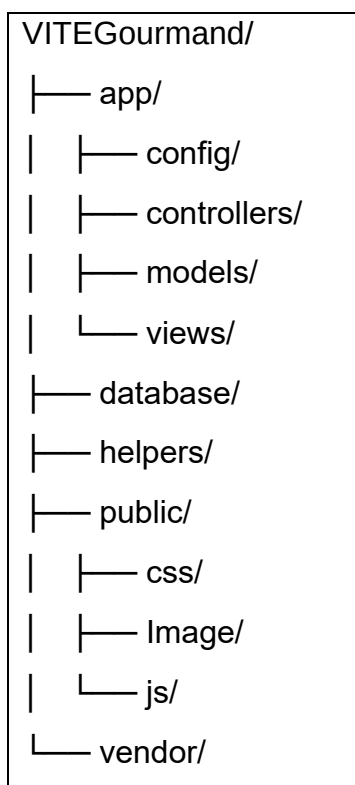
```

20  if (file_exists($localConfigPath)) {
21      require $localConfigPath;
22  } else {
23      die("Config locale manquante :
        copie constants.local.php.example
        vers constants.local.php et
        renseigne tes identifiants.");
24  }
25

```

Ce même fichier `app/config/constants.php` gère aussi la bascule automatique entre l'environnement local (XAMPP, `constants.local.php`) et l'environnement de production Heroku (variable `JAWSDB_URL` fournie par l'add-on JawsDB) — le détail de ce mécanisme est repris en section 5 (Déploiement).

2.4 Arborescence du projet



2.5 Installation locale

La procédure d'installation complète (clonage, composer install, création de la base

MySQL et import de `schema.sql` + `fixtures.sql`, configuration de `.env` et `constants.local.php`, démarrage d'Apache/MySQL, accès via `http://localhost/VITEGourmand/public/`) est détaillée pas à pas dans le `README.md` du dépôt, pour rester la seule source à maintenir en cas d'évolution des étapes.

Les comptes de test permettant de suivre les différents parcours (client, employé, administrateur) sont fournis dans le manuel d'utilisation, et non dans cette documentation technique ni dans le `README`.

3.1 Présentation du MCD (MySQL)

Le modèle relationnel s'organise autour de trois grands pôles : les utilisateurs, la carte (plats/menus) et les commandes.

Entités de référence (sans dépendance) : role (rôles utilisateur), theme et regime (catégorisation des menus), horaire (horaires d'ouverture), allergene (14 allergènes majeurs, seedés conformément au règlement UE n° 1169/2011).

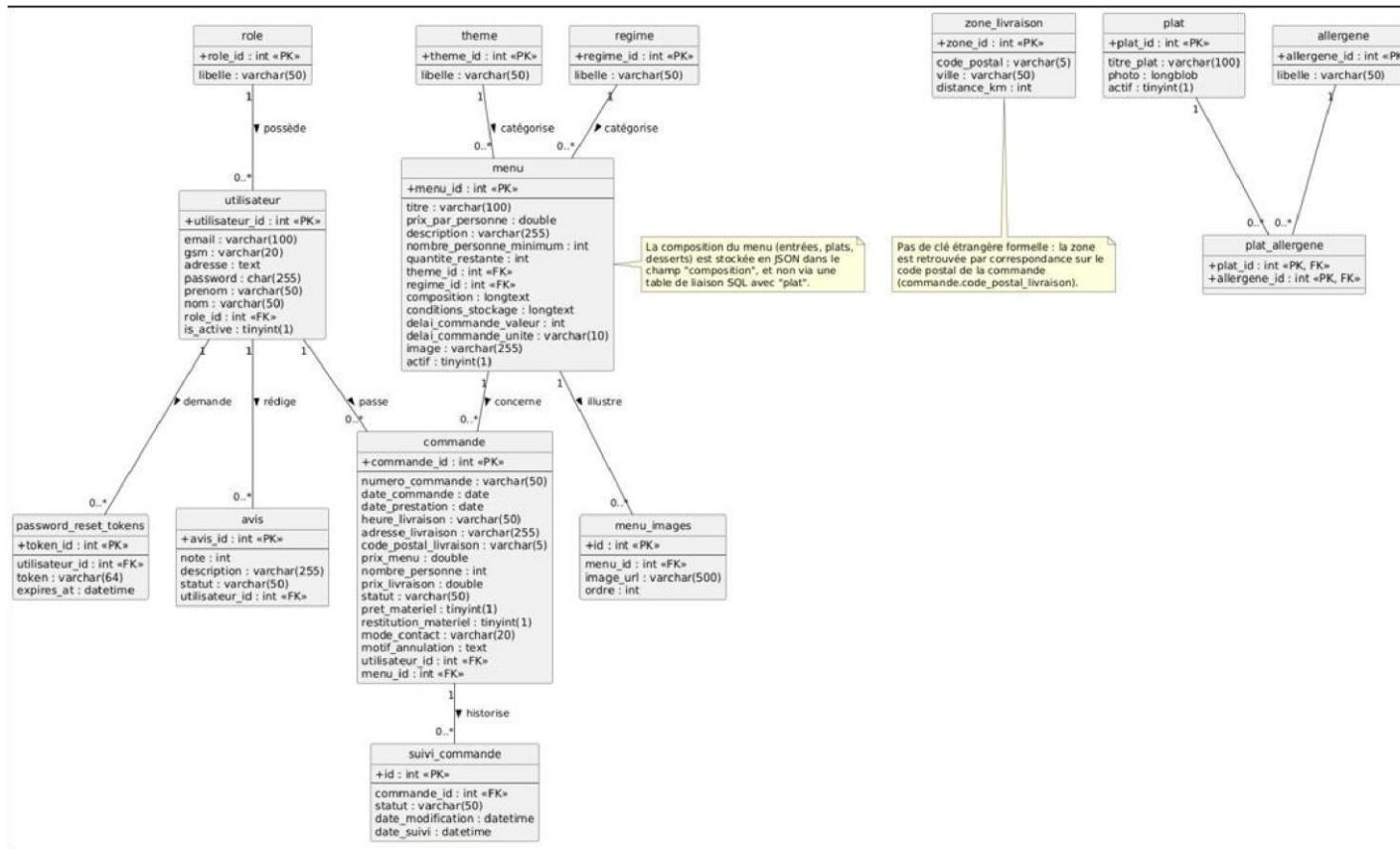
Utilisateurs : la table utilisateur porte un role_id (clé étrangère vers role), ce qui permet de distinguer client / employé / administrateur sans dupliquer de table par rôle.

Carte : un plat peut posséder plusieurs allergene et un allergène concerne plusieurs plats relation N-N portée par la table de jointure plat_allergene. Un menu est rattaché à un theme et à un regime, et sa composition (entrée/plat/dessert) est stockée dans une colonne JSON (composition), avec une contrainte CHECK (json_valid(...)) qui garantit que seul du JSON valide peut y être écrit un exemple concret d'usage raisonné du JSON à l'intérieur même de MySQL, sans pour autant sortir du modèle relationnel. Chaque menu peut avoir plusieurs images de galerie (menu_images, avec un champ ordre pour l'affichage), et porte également un délai de commande minimum (delai_commande_valeur / delai_commande_unite).

Commandes : une commande référence un utilisateur et un menu, et son évolution de statut (« En attente » → « Acceptée » → ... → « Terminée » ou « Annulée ») est historisée dans suivi_commande, plutôt que d'écraser le statut précédent — ce qui permet de reconstituer l'historique complet d'une commande.

Cardinalités principales :

- role (1) — (N) utilisateur
- theme (1) — (N) menu, regime (1) — (N) menu
- plat (N) — (N) allergene via plat_allergene
- menu (1) — (N) menu_images
- utilisateur (1) — (N) commande, menu (1) — (N) commande
- commande (1) — (N) suivi_commande
- utilisateur (1) — (N) password_reset_tokens



. La table avis existe encore côté MySQL mais n'est plus utilisée : les avis client sont désormais exclusivement stockés dans MongoDB. Cette table est un reliquat d'une version antérieure du modèle, conservée telle quelle plutôt que supprimée en cours de projet pour ne pas risquer de casser une dépendance non identifiée en fin de sprint.

La table zone_livraison n'est plus interrogée par le code non plus : le calcul de zone de livraison est désormais fait dynamiquement via l'API externe api.adresse.data.gouv.fr et une formule de distance (Haversine), voir [app/models/ZoneLivraison.php](#). Cette table correspondait à une première approche (zones fixes définies par code postal), remplacée en cours de projet par la géolocalisation par API (commit 79764e4) une fois cette solution jugée plus fiable et plus simple à maintenir qu'une liste de zone à tenir à jour manuellement. Elle a été volontairement laissée dans le schéma plutôt que supprimée, pour garder une trace du cheminement technique ayant mené au choix définitif.

3.2 Complément NoSQL (MongoDB Atlas)

Deux collections MongoDB complètent le modèle relationnel, pour les données à schéma variable (voir aussi section 1.2) :

- avis : { user_id, commande_id, nom_client, note, commentaire, statut, date_creation } — un avis par commande, avec un statut de modération (en_attente, valide, refuse) géré par l'employé/administrateur.
- stats_commandes (via StatsCommandeModel) : statistiques agrégées par commande, utilisées pour le tableau de bord administrateur.

Il n'y a pas de clé étrangère technique entre MySQL et MongoDB (ce sont deux bases distinctes) : le lien se fait au niveau applicatif via les identifiants utilisateur_id / commande_id, stockés comme simples valeurs dans les documents MongoDB.

3.3 Diagramme de classes — architecture MVC

L'application suit un modèle MVC classique :

- Vue (app/views/*.view.php)↑ affiche les données transmises par Contrôleur
- (app/controllers/*Controller.php)↓interroge / met à jour via Modèle (app/models/*.php)
- persiste via PDO (MySQL) ou driver mongodb/mongodb (MongoDB Atlas)

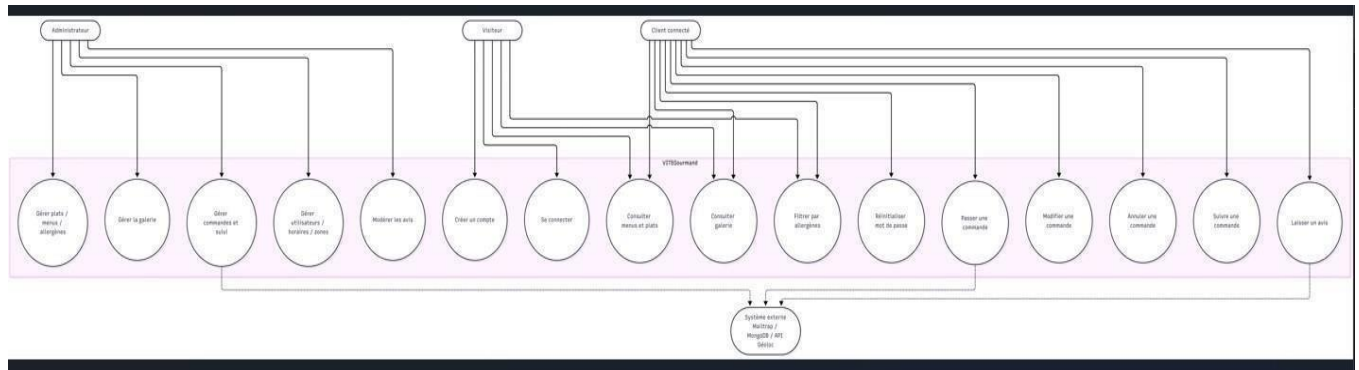
Chaque modèle encapsule l'accès à une seule ressource (un modèle = une table, ou une collection pour AvisModel/StatsCommandeModel), ce qui évite qu'un contrôleur n'exécute de requête SQL/Mongo directement — la logique d'accès aux données reste centralisée et réutilisable entre plusieurs contrôleurs.

4. Diagramme de cas d'utilisation et diagrammes de séquence

4.1 Acteurs

- Visiteur utilisateur non connecté (consultation uniquement).
- Client utilisateur connecté (commande, suivi, avis).
- Administrateur back-office (gestion de la carte, des commandes, des utilisateurs). Par souci de simplicité, ce document regroupe sous cet unique acteur les deux rôles réellement présents dans le code (role_id 1 = Admin, role_id 2 = Employé) qui partagent la majorité des cas d'usage de back-office (gestion des commandes, modération des avis) — seule la gestion des comptes employés est strictement réservée au rôle Admin.
- Système les services externes sollicités par l'application (Mailtrap pour l'envoi d'emails, MongoDB Atlas pour les avis/statistiques, API adresse.data.gouv.fr pour la géolocalisation).

4.2 Diagramme de cas d'utilisation

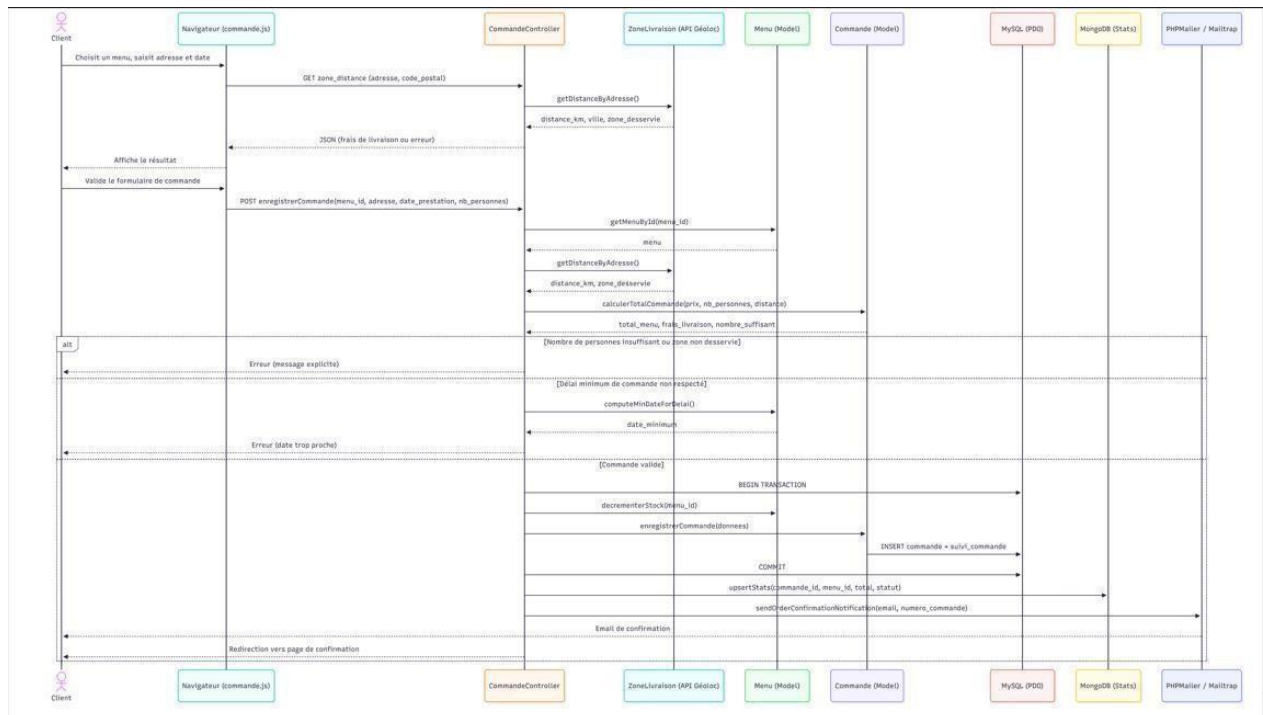


Cas d'utilisation par acteur (contenu du diagramme, pour référence)

- Visiteur : Consulter menus et plats · Consulter galerie · Filtrer par allergènes · Créer un compte.
- Client connecté : Se connecter · Réinitialiser mot de passe · Passer une commande · Modifier une commande · Annuler une commande · Suivre une commande · Laisser un avis.
- Administrateur : Gérer plats / menus / allergènes · Gérer la galerie · Gérer commandes et suivi · Gérer utilisateurs / horaires / zones · Modérer les avis.
- Système externe (Mailtrap / MongoDB / API Géoloc) : sollicité en arrière-plan par les cas d'usage ci-dessus (envoi d'emails, stockage des avis/statistiques, calcul de zone de livraison).

4.3 Diagramme de séquence « Passer une commande »

Cette séquence correspond au parcours implémenté dans commandeController.php (fonction enregistrerCommande) : vérification de la zone de livraison, calcul du total, vérification du délai de commande minimum, puis enregistrement transactionnel de la commande.



Déroulé de la séquence

Participants : Client · Navigateur (commande.js) · CommandeController · ZoneLivraison (API Géoloc) · Menu (Model) · Commande (Model) · MySQL (PDO) · MongoDB (Stats) · PHPMailer / Mailtrap.

1. Le Client choisit un menu, saisit une adresse et une date de prestation.
2. Navigateur → CommandeController : GET zone_distance(adresse, code_postal)
3. CommandeController → ZoneLivraison : getDistanceByAdresse()
4. ZoneLivraison → CommandeController : distance_km, ville, zone_desservie
5. CommandeController → Navigateur : réponse JSON (frais de livraison ou erreur), affichée au Client.
6. Le Client valide le formulaire de commande.
7. Navigateur → CommandeController : POST enregistrerCommande(menu_id, adresse, date_prestation, nb_personnes)

8. CommandeController → Menu : getMenuById(menu_id) → menu
9. CommandeController → ZoneLivraison : getDistanceByAdresse() → distance_km, zone_desservie
10. CommandeController : calculerTotalCommande(prix, nb_personnes, distance) → total_menu, frais_livraison, nombre_suffisant

alt : Nombre de personnes insuffisant ou zone non desservie
Retour d'une erreur explicite au Navigateur ; la commande n'est pas enregistrée. alt : Délai minimum de commande non respecté
CommandeController appelle computeMinDateForDelai() → date_minimum ; si la date choisie est trop proche, une erreur est renvoyée (date trop proche).
alt : Commande valide

1. BEGIN TRANSACTION (MySQL)
2. Commande → Menu : decrementsStock(menu_id)
3. CommandeController → Commande : enregistrerCommande(donnees) → INSERT commande + suivi_commande
4. COMMIT
5. CommandeController → MongoDB : upsertStats(commande_id, menu_id, total, statut)
6. CommandeController → PHPMailer/Mailtrap : sendOrderConfirmationNotification(email, numero_commande) → email de confirmation envoyé
7. CommandeController → Navigateur : redirection vers la page de confirmation, affichée au Client

⚠ À noter : la vérification de zone de livraison est faite deux fois (côté navigateur pour l'affichage immédiat, puis côté serveur avant tout enregistrement) c'est volontaire, la validation côté client seule ne suffisait jamais à garantir la fiabilité d'une donnée envoyée au serveur.

5. Déploiement Heroku démarche et étapes

5.1 Démarche générale

L'application est déployée sur Heroku (application vitegourmand-nicolas-ecf, région eu), via le buildpack officiel heroku/php, détecté automatiquement grâce à la présence de composer.json à la racine. Le déploiement se fait par un simple git push heroku main :

Heroku installe les dépendances Composer et démarre le serveur selon les instructions du Procfile (web: heroku-php-apache2 public/).

Le choix de la région eu plutôt que la région américaine par défaut est délibéré : le public visé (traiteur événementiel basé à Bordeaux) est européen, et héberger les données dans l'Union européenne est cohérent avec les exigences RGPD évoquées en 1.6.

Deux contraintes propres à Heroku, différentes de l'environnement XAMPP local, ont dû être anticipées :

- Le système de fichiers de Heroku est Linux, sensible à la casse (contrairement à Windows/XAMPP en développement) : un nom de fichier ou un require mal cassé fonctionne en local mais plante en production.
- La base MySQL n'est pas fournie nativement : elle passe par l'add-on JawsDB, dont l'URL de connexion est injectée automatiquement dans une variable d'environnement (JAWSDB_URL), différente du couple DB_USER/DB_PASS utilisé en local.

Le dépôt local dispose de deux remotes Git distincts, sans interaction entre eux : origin (GitHub, dépôt public livré pour l'ECF) et heroku (ajouté automatiquement par heroku create, utilisé uniquement pour déployer le code).

5.2 Étapes suivies jusqu'au premier déploiement réussi

Étape 1 Création de l'application Heroku

```
heroku create vitegurmand-nicolas-ecf --region eu
```

Le nom d'application est unique sur toute la plateforme Heroku (tous comptes confondus), d'où le suffixe distinctif. L'URL publique obtenue (<https://vitegurmandnicolasecf<id>.herokuapp.com/>) affiche une page par défaut tant qu'aucun code n'est déployé.

Étape 2 Base MongoDB (NoSQL)

MongoDB étant déjà hébergé sur MongoDB Atlas (cloud) et non en local (voir 1.2), aucune migration de données n'est nécessaire avant le déploiement seule la configuration réseau doit être vérifiée : les dynos Heroku n'ayant pas d'adresse IP fixe, l'accès réseau du cluster Atlas doit autoriser les connexions depuis n'importe quelle IP (0.0.0.0/0), sans quoi Heroku ne peut pas s'y connecter.

Par précaution, le mot de passe du compte Atlas a été régénéré avant d'être reporté dans les Config Vars Heroku, celui-ci ayant transité en clair dans un échange de travail même réflexe de prudence que celui appliqué plus loin après l'exposition accidentelle d'identifiants (voir Étape 7).

Étape 3 État du dépôt Git avant déploiement

Le flux Git suivi avant ce déploiement respecte le fonctionnement attendu par le sujet ECF (branche principale + branche de développement) : développement et commit des fichiers schema.sql/fixtures.sql sur dev, push vers origin/dev, puis merge de dev dans main une fois la base considérée stable (fast-forward, sans conflit), et c'est uniquement le contenu de main qui part ensuite vers Heroku via git push heroku main.

Étape 4 Base de données relationnelle (MySQL) sur Heroku

```
heroku addons:create jawsdb -a vitegurmand-nicolas-ecf
```

L'add-on attribue automatiquement le plan gratuit jawsdb:kitefin, une base provisionnée sur AWS RDS en région eu-west-1 (Irlande), cohérente avec le choix de région de l'application. Heroku génère automatiquement une Config Var JAWSDB_URL contenant l'ensemble des identifiants de connexion sous forme d'URL unique.

Deux incidents rencontrés à cette étape, résolus en cours de route :

- Incompatibilité du client MySQL local : la tentative d'import de `schema.sql/fixtures.sql` via le client `mysql.exe` fourni par XAMPP a échoué (Plugin `caching_sha2_password` could not be loaded). Cause : ce client ne sait pas parler le protocole d'authentification par défaut de MySQL 8 utilisé par JawsDB/AWS RDS une incompatibilité côté client, pas côté serveur. Contournement retenu : un import via un script PHP utilisant PDO (le driver `mysqlnd` de PHP gère nativement `caching_sha2_password`), plutôt que d'installer en urgence un nouveau client MySQL un choix cohérent avec la stack déjà utilisée par l'application (PHP + PDO).
- Connexion à la base non préparée pour la production : `app/config/constants.php` contenait encore uniquement la configuration locale XAMPP en dur. Correction appliquée : le fichier détecte désormais la présence de la Config Var `JAWSDB_URL` et, si elle existe, en extrait l'hôte, la base, l'utilisateur et le mot de passe via `parse_url()` ; sinon il conserve la configuration locale par défaut (voir le détail du code en 5.4). Une seule base de code fonctionne ainsi en local et en production, sans variable supplémentaire à gérer côté MySQL.

Étape 5 Variables d'environnement (Config Vars Heroku)

Les identifiants Mailtrap (`MAILTRAP_USERNAME`, `MAILTRAP_PASSWORD`, `MAILTRAP_HOST`, `MAILTRAP_PORT`) et MongoDB Atlas (`MONGO_USERNAME`, `MONGO_PASSWORD`, `MONGO_CLUSTER`, `MONGO_APPNAME`, `MONGO_DB_NAME`) ont été repris à l'identique du `.env` local via `heroku config:set`.

La variable MySQL (`JAWSDB_URL`) était déjà configurée automatiquement par l'add-on (Étape 4). L'application dispose ainsi de tous les identifiants nécessaires sans qu'aucun ne soit écrit en dur dans le code ni committé sur GitHub.

Ce découpage en deux canaux de configuration distincts est un choix assumé, et non une simplification : `.env` (Mailtrap + MongoDB) et le fichier dédié à la base relationnelle (`constants.local.php` en local, `JAWSDB_URL` en production) suivent le même principe un secret, un fichier/canal par type de source de données plutôt que de tout regrouper dans un seul fichier `.env`.

Cela permet de démarquer clairement la base relationnelle des autres services externes, et de faire évoluer ou tourner les identifiants de l'une sans toucher aux autres.

Ce même niveau de rigueur a in fine été appliqué aux trois sources de secrets du projet, bien qu'à des moments différents : rotation préventive du mot de passe Atlas avant même le premier déploiement (Étape 2), rotation des identifiants JawsDB en réponse à l'incident de sécurité de l'Étape 7, et durcissement de la configuration MySQL locale (XAMPP) en séparant les identifiants dans `constants.local.php` gitigné (voir 5.3, `da08c0d`).

Étape 6 Procfile et composer.json

Le Procfile était absent à la racine du dépôt : il a été créé avec `web: heroku-phpapache2 public/`, pour que Heroku serve l'application depuis le dossier `public/` uniquement (le reste du code `app/`, `database/`, etc. n'est pas exposé publiquement). `composer.json` ne déclarait par ailleurs que la librairie `mongodb/mongodb`, pas l'extension PHP native correspondante : ajout de `ext-mongodb` et `ext-pdo_mysql` en dépendances de plateforme, pour que le buildpack PHP de Heroku les active au déploiement.

Un arbitrage de sécurité assumé a été documenté à cette étape : `mongodb/mongodb` en version

2.3.0 (seule version de la ligne 2.3.x) fait l'objet d'une alerte de sécurité Composer (réf. PKSA61k5-cqr9-b8b4), corrigée en version 2.4+, laquelle exige une extension native plus récente que celle installée en local.

Faute de temps avant la deadline ECF, la version 2.3.0 a été conservée plutôt que de risquer une mise à jour d'extension en urgence, le risque étant jugé limité : l'usage de MongoDB est restreint aux statistiques de l'espace administrateur, authentifié et non exposé aux visiteurs. Mise à jour identifiée comme à faire après le rendu (voir 5.6).

Étape 7 Premier déploiement réussi

```
git push heroku main
```

Build terminé sans erreur (stack Heroku-24, PHP 8.5.10, extension MongoDB 2.4.1, Apache 2.4.68), dépendances Composer installées depuis le lock file, Procfile détecté. Application en ligne à l'issue de ce déploiement.

Incident de sécurité identifiants exposés sur le dépôt public

Le script `import_heroku.php`, créé à l'Étape 4 pour contourner l'incompatibilité du client MySQL local, contenait les identifiants de connexion JawsDB en clair. Il devait être supprimé après usage mais a été inclus par erreur dans un commit via `git add -A`, puis poussé sur `origin/dev` et `origin/main` donc exposé publiquement sur GitHub.

Mesures correctives appliquées, dans cet ordre :

1. Suppression du fichier du dépôt (nouveau commit).
2. Ajout d'une règle `.gitignore` (`import_*.php`) pour empêcher toute réapparition d'un fichier de ce type.
3. Rotation complète des identifiants compromis : destruction puis recréation de l'add-on JawsDB (nouvelle base, nouveaux identifiants générés automatiquement par Heroku, réimport de `schema.sql` et `fixtures.sql`), rendant les identifiants exposés définitivement inutilisables.

Cet incident, bien que fâcheux, illustre une réponse à incident de sécurité complète (détection, correction, rotation) documenté tel quel plutôt que passé sous silence, dans le même esprit que les écarts de pratique Git assumés ailleurs dans cette copie.

Étape 8 Correction des chemins en dur et redéploiement

Après ce premier déploiement, la mise en page Bootstrap n'était pas appliquée et les liens de navigation renvoyaient des pages « Not Found ». Cause : tous les liens et chemins CSS/JS étaient écrits en dur avec le préfixe /VITEGourmand/public (structure de l'URL locale sous XAMPP), inexistant sur Heroku où l'application est servie à la racine du domaine.

Correction : ajout d'une constante `BASE_URL` dans `constants.php` (vide en production via la détection de `JAWSDB_URL`, égale à `/VITEGourmand/public` en local), puis mise à jour des fichiers concernés (en-tête, pied de page, vue de connexion, réinitialisation de mot de passe, emails). Après redéploiement : mise en page et navigation opérationnelles.

5.3 Corrections supplémentaires identifiées lors des tests suivants

Les vérifications menées après l'Étape 8 (voir 5.5) ont fait remonter d'autres erreurs propres à l'environnement de production, corrigées au fur et à mesure :

Commit	Problème rencontré / action
60851ec	Crash en production sur /menus : <code>mb_strtolower()</code> (utilisée dans <code>menusController.php</code>) provoquait une erreur fatale car <code>ext-mbstring</code> n'était pas déclarée dans <code>composer.json</code> , donc non activée par le buildpack. Correction : ajout de <code>"ext-mbstring": "*" </code> .
fd4eb27/2ac6ff6	Épingleage de la version PHP dans <code>composer.json</code> puis régénération de <code>composer.lock</code> (le hash de contenu du lock file doit rester cohérent avec <code>composer.json</code> , sans quoi Heroku refuse le build).
59a9364/6289164	Nouveau crash dû à la sensibilité à la casse du filesystem Linux : <code>ReviewController.php</code> (casse initiale) ne correspondait pas au nom utilisé dans les require d' <code>index.php</code> (<code>reviewController.php</code>). Fonctionnait en local (Windows insensible à la casse), plantait sur Heroku. Renommage du fichier pour faire correspondre exactement la casse.
da08c0d	Durcissement complémentaire, a posteriori : extraction des identifiants MySQL locaux restants dans le code vers <code>constants.local.php</code> (gitignoré), distinct de la correction JawsDB déjà faite en production dès l'Étape 4.

Cette liste, comme l'incident de l'Étape 7, illustre une réalité du déploiement : plusieurs erreurs n'apparaissent qu'en production (casse des fichiers, extensions PHP manquantes) et ne sont pas détectables en local sous XAMPP/Windows d'où l'intérêt de documenter la démarche de correction plutôt que de ne présenter qu'un déploiement idéalisé,

5.4 Bascule automatique entre configuration locale et production

Le code ne contient aucune branche if (environnement de prod) à maintenir manuellement :

app/config/constants.php détecte simplement la présence de la variable JAWSDB_URL, fournie uniquement par Heroku :

```
$jawsdbUrl = getenv('JAWSDB_URL');

if ($jawsdbUrl) {
    // Production Heroku : identifiants extraits de l'URL JawsDB
    $dbParts = parse_url($jawsdbUrl);
    define('DB_HOST', $dbParts['host']);    define('DB_NAME',
    ltrim($dbParts['path'], '/'));    define('DB_USER',
    $dbParts['user']);
    define('DB_PASS', $dbParts['pass']);
} else {
    // Local XAMPP : identifiants lus depuis constants.local.php
    ...
}
```

Côté MongoDB, aucune bascule n'est nécessaire : l'application utilise MongoDB Atlas (cloud) aussi bien en local qu'en production, via les mêmes variables MONGO_* (voir 1.2 et 2.3), avec le réglage réseau 0.0.0.0/0 décrit en 5.2 (Étape 2).

5.5 Vérifications post-déploiement

Après chaque déploiement, un parcours de vérification manuel a été suivi : connexion avec un compte administrateur, ajout d'un menu/plat avec photo, consultation de /menus (page ayant été source d'un crash, voir 5.3), passage d'une commande complète, affichage des statistiques MongoDB, réinitialisation de mot de passe via Mailtrap, et lecture de heroku logs -tail pour confirmer l'absence d'erreur PHP silencieuse.

5.6 Limites actuelles et améliorations possibles

- mongodb/mongodb reste en version 2.3.0 malgré l'alerte de sécurité PKSA-61k5cqr9b8b4 (voir 5.2, Étape 6) mise à jour vers 2.4+ à faire après le rendu ECF.
- Il n'existe pas de script de migration automatisé pour appliquer `schema.sql/fixtures.sql` sur la base de production : l'import reste manuel (voir l'incident de l'Étape 4), ce qui est un point d'amélioration identifié pour un projet destiné à évoluer.
- Pas d'intégration continue (CI) : les vérifications post-déploiement (5.5) sont manuelles. Une CI GitHub Actions pourrait les automatiser avant tout déploiement.
- La stack Heroku utilisée (Heroku-24) n'est plus la plus récente (Heroku-26 disponible) mise à niveau non bloquante, envisageable après le rendu.
- Heroku ayant supprimé son offre gratuite, une migration vers une alternative (Render, Fly.io) ou une conteneurisation Docker serait à envisager pour la suite du projet au-delà du cadre de cet ECF.